Machine Project 2 - Color Transformations

IMAGPRO - S11

Group Members:
- Agustines, Alfred Bastin
- De Leon, Allan David
- Depasucat, Michael Angelo
- Marquez, Vincent Alvin
- Padilla, Kai Hiori

Import Dependencies and Configurations

In [125]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
import random
from glob import glob
import matplotlib.colors as mcolors

# ==============================
# Directory Configuration (Relative Paths)
# ==============================
# Each action item writes to its own folder so outputs can be compared
side-by-side.

DATASET_DIR      = 'dataset/flowers_dataset'
TRANSFORMED_DIR  = 'A1_transformed'           # Item 1 output
CLEANED_DIR      = 'A2_cleaned'               # Item 2 output
ENHANCED_EXP_DIR = 'A3_enhanced/exposure'     # Item 3 output — gamma
correction only
ENHANCED_SAT_DIR = 'A3_enhanced/saturation'   # Item 3 output — gamma +
saturation (final)
BASE_AUG_DIR     = 'A4_augmented/exposure'    # Item 4 output
BASE_AUG_DIR2    = 'A4_augmented/saturation'  # Item 4 output

for d in [TRANSFORMED_DIR, CLEANED_DIR, ENHANCED_EXP_DIR,
ENHANCED_SAT_DIR]:
    os.makedirs(d, exist_ok=True)

aug_subfolders = ['patch', 'shift', 'rotate', 'flip', 'sat_increase',
'sat_decrease']
for folder in aug_subfolders:
    os.makedirs(os.path.join(BASE_AUG_DIR, folder), exist_ok=True)
    os.makedirs(os.path.join(BASE_AUG_DIR2, folder), exist_ok=True)

# ==============================
# Augmentation Parameters (from MP1)
# ==============================
P_SIZE      = (25, 25)        # Black patch size (width, height)
S_TX, S_TY  = 20, 20          # Shift offsets
ROT_A       = 30              # Rotation angle (degrees)
FLIP_A      = 'horizontal'    # Flip mode

# ==============================
# Enhancement Parameters (Manual Selection)
# ==============================
# After running the "Inspect Dataset Stats" cell, set these to the
# filenames of the brightest and most saturated images in A2_cleaned/
BRIGHTEST_IMAGE      = '0020.jpg'   # <-- update after inspecting stats
MOST_SATURATED_IMAGE = '0003.jpg'   # <-- update after inspecting stats

# Saturation augmentation scale factors
```

```
SAT_INCREASE = 1.5   # 50% saturation boost
SAT_DECREASE = 0.5   # 50% saturation reduction
```

Action Item 1: Information Extraction

Given the image dataset:
- Reshape the images to `(100, 100, 3)`
- Save the transformed images as JPEG files in a separate directory
- Create a histogram for the following channels: **Reds, Greens, Blues, Hues, Saturations, Intensities**

In [126]:

```
# Collect all images from the dataset folder
image_extensions = ['*.jpg', '*.jpeg', '*.png', '*.jfif']
image_files = []
for ext in image_extensions:
    image_files.extend(glob(os.path.join(DATASET_DIR, ext)))
image_files = sorted(image_files)

formatted_images = []
failed_images = []

for fpath in image_files:
    img = cv2.imread(fpath)
    if img is not None:
        resized = cv2.resize(img, (100, 100))    # Reshape to (100, 100, 3)
        fname = os.path.splitext(os.path.basename(fpath))[0] + '.jpg'
        cv2.imwrite(os.path.join(TRANSFORMED_DIR, fname), resized)
        formatted_images.append(resized)
    else:
        failed_images.append(os.path.basename(fpath))

print(f"Successfully formatted {len(formatted_images)} images → saved to
'{TRANSFORMED_DIR}'")
if failed_images:
    print(f"Failed to read {len(failed_images)} images: {failed_images}")
```

```
Successfully formatted 10 images → saved to 'A1_transformed'
```

Action Item 1 (continued): Channel Histograms

Aggregate histograms are computed across all images for the six channels:
- **RGB space:** Red, Green, Blue
- **HSV space:** Hue, Saturation, Value (Intensity)

> OpenCV stores Hue in the range `[0, 179]` — multiply by 2 for the standard `[0°, 360°]` scale.

In [ ]:

```
agg = {ch: np.zeros(256 if ch != 'H' else 180) for ch in ['R', 'G', 'B',
'H', 'S', 'V']}

for img in formatted_images:
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    agg['R'] += cv2.calcHist([img], [2], None, [256], [0, 256]).flatten()
    agg['G'] += cv2.calcHist([img], [1], None, [256], [0, 256]).flatten()
    agg['B'] += cv2.calcHist([img], [0], None, [256], [0, 256]).flatten()
    agg['H'] += cv2.calcHist([hsv],  [0], None, [180], [0, 180]).flatten()
    agg['S'] += cv2.calcHist([hsv],  [1], None, [256], [0, 256]).flatten()
    agg['V'] += cv2.calcHist([hsv],  [2], None, [256], [0, 256]).flatten()

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
fig.suptitle('Aggregate Channel Histograms (All Images)', fontsize=14,
fontweight='bold')

channel_cfg = [
    ('R', 'Reds',        'red'),
    ('G', 'Greens',      'green'),
    ('B', 'Blues',       'blue'),
    ('H', 'Hues',        'purple'),
    ('S', 'Saturations', 'orange'),
    ('V', 'Intensities', 'gray'),
]

for ax, (key, title, color) in zip(axes.flatten(), channel_cfg):
    hist = agg[key]
    ax.plot(hist, color=color, linewidth=1.2)
    ax.fill_between(range(len(hist)), hist, alpha=0.25, color=color)
    ax.set_title(title, fontsize=12)
    ax.set_xlabel('Pixel Value')
    ax.set_ylabel('Frequency')
    ax.set_xlim([0, len(hist)])
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
print("Note: Hue values in OpenCV are halved (0–179); multiply by 2 for
standard 0–360° scale.\n")


# RGB Computation
rgb_results = {
    'Red':   np.average(np.arange(256), weights=agg['R']),
    'Green': np.average(np.arange(256), weights=agg['G']),
    'Blue':  np.average(np.arange(256), weights=agg['B'])
}

# HSV Computation
# Finds the indices (0-179) with the highest counts
top_3_indices = np.argsort(agg['H'])[-3:][::-1]

print("--- RGB Space Figures ---")
```

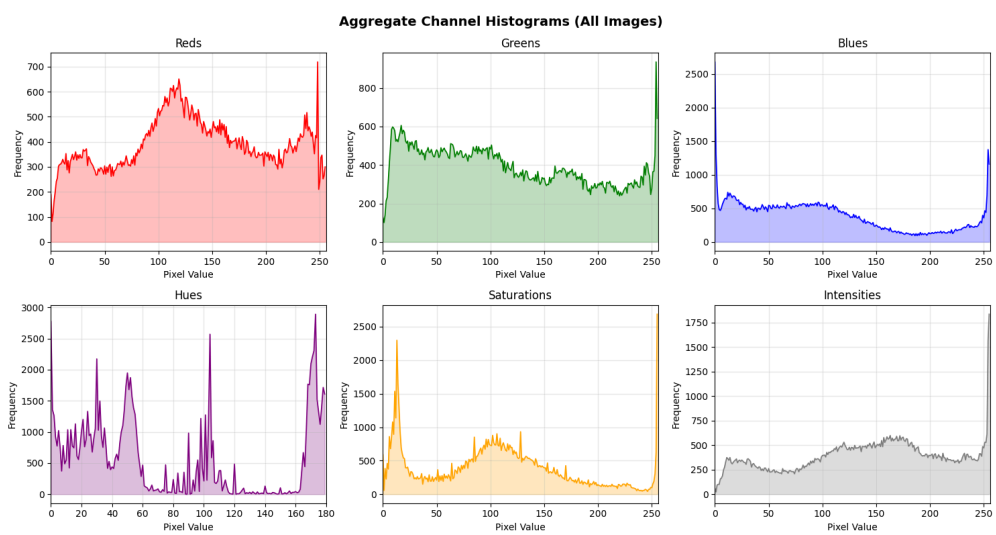
```
for channel, value in rgb_results.items():
    print(f"Mean {channel} Intensity: {value:.2f}")

print("\n--- HSV Space Figures ---")
print("Top 3 Hues (Most frequent to least frequent):")
for i, idx in enumerate(top_3_indices, 1):
    degree = idx * 2
    count = agg['H'][idx]
    print(f"{i}. Hue: {degree}° (Frequency count: {int(count)})")

# Visualizes the HSV Figures
degrees = top_3_indices*2
hsv_colors = [(d / 360.0, 1.0, 1.0) for d in degrees]
rgb_colors = [mcolors.hsv_to_rgb(hsv) for hsv in hsv_colors]

fig, ax = plt.subplots(figsize=(8, 3))
for i, color in enumerate(rgb_colors):
    ax.add_patch(plt.Rectangle((i, 0), 0.8, 1, color=color))
    ax.text(i + 0.4, -0.15, f"{degrees[i]}°", ha='center', fontsize=12,
fontweight='bold')

ax.set_xlim(-0.2, len(degrees))
ax.set_ylim(-0.2, 1.2)
ax.set_aspect('equal')
ax.axis('off')
plt.title("HSV Hue Visualization", fontsize=14)
plt.show()
```

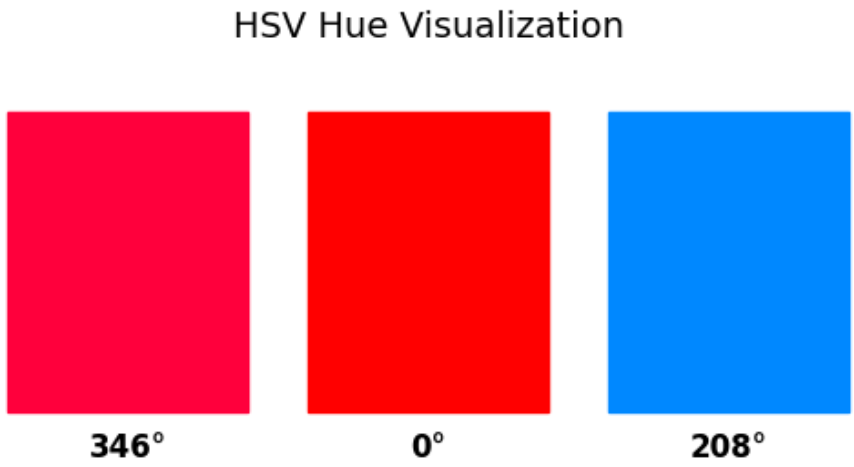



```
Note: Hue values in OpenCV are halved (0–179); multiply by 2 for
standard 0–360° scale.

--- RGB Space Figures ---
Mean Red Intensity: 132.37
Mean Green Intensity: 116.45
Mean Blue Intensity: 97.31

--- HSV Space Figures ---
Top 3 Hues (Most frequent to least frequent):
```

```
1. Hue: 346° (Frequency count: 2892)
2. Hue: 0° (Frequency count: 2778)
3. Hue: 208° (Frequency count: 2569)
```



Guide Questions — Action Item 1:

1. In the RGB space, which channel is most likely to be observed for all images?
- In the RGB space, the red channel is the most dominant across all images, as supported by the extracted mean intensity values where red (132.37) is higher than green (116.45) and blue (97.31). This indicates that, on average, pixels in the dataset contain stronger red components compared to the other channels. Since RGB is an additive color model, higher channel intensity corresponds to a stronger presence of that color in the image. The dominance of red is consistent with the nature of the dataset, majority of which contains pink flowers, as pink is primarily composed of high red values combined with smaller amounts of blue and minimal green. This aligns with how RGB represents color through varying intensities of red, green, and blue light, where increasing red intensity directly contributes to reddish or pinkish tones *(Difference Between RGB, CMYK, HSV, and YIQ Color Models, 2021)*. Therefore, both the statistical data and the expected color composition confirm that the red channel is the most prominent.

2. In the HSV space, which top three hues are most likely to be observed for all images?
- In the HSV space, the three most frequently observed hues based on the extracted statistics are 346°, 0°, and 208°. These correspond respectively to magenta/pink, red, and a blue-cyan region on the color wheel. In the HSV model, hue is represented as an angle from 0° to 360°, where 0° corresponds to red and values near 300°–360° correspond to magenta *(HSV (Hue, Saturation, and Value), n.d.) (HSV Color Model in Computer Graphics, 2022)*. The presence of 346° indicates a strong concentration of pink or magenta tones, which is expected given the dataset of pink flowers. The second most frequent hue, 0°, reinforces the dominance of red tones, likely due to highlights or variations in petal coloration. The third hue, 208°, falls within the blue-cyan range, which can be attributed to background elements such as sky, shadows, or foliage contrast. Since hue values are distributed along a circular spectrum, it is common for natural images to exhibit multiple dominant hues due to variations in lighting and scene composition. Thus, the observed hues: magenta (346°), red (0°), and blue-cyan (208°), accurately reflect both the subject (pink flowers) and the environmental context of the images.

Action Item 2: Data Cleaning

Develop a function that attains the following:
- Remove all images that are taken **during the night**
- Remove all images that are **not pink flowers**

In [ ]:

```
# ==============================
# Night Detection
# ==============================
# Strategy: images taken at night have a very low overall brightness.
NIGHT_V_THRESHOLD = 85

def is_night_image(img_bgr, threshold=NIGHT_V_THRESHOLD):
    """Return True if the image is likely taken at night (low mean
brightness)."""
    hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
    return float(np.mean(hsv[:, :, 2])) < threshold

# ==============================
# Pink Flower Detection
# ==============================
# Strategy: pink/magenta hues in OpenCV HSV occupy H ∈ [140, 175]
# (equivalent to ~280–350° on the standard 0–360° scale).
# If the fraction of qualifying "pink" pixels exceeds 5% of the image area,
# the image is classified as containing a pink flower.
HUE_LOW, HUE_HIGH = 140, 175
SAT_MIN           = 40
PINK_RATIO_MIN    = 0.05

def is_pink_flower(img_bgr, hue_low=HUE_LOW, hue_high=HUE_HIGH,
                   sat_min=SAT_MIN, min_ratio=PINK_RATIO_MIN):
    """Return True if the image contains a sufficient proportion of pink
pixels."""
    hsv   = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
    lower = np.array([hue_low, sat_min, 30])
    upper = np.array([hue_high, 255, 255])
    mask  = cv2.inRange(hsv, lower, upper)
    ratio = np.sum(mask > 0) / (img_bgr.shape[0] * img_bgr.shape[1])
    return ratio >= min_ratio

# ==============================
# Apply Cleaning to Transformed Dataset
# ==============================
transformed_files = sorted(glob(os.path.join(TRANSFORMED_DIR, '*.jpg')))
kept, removed_night, removed_non_pink = [], [], []

for fpath in transformed_files:
    img   = cv2.imread(fpath)
    fname = os.path.basename(fpath)

    if is_night_image(img):
        removed_night.append(fname)
        continue

    if not is_pink_flower(img):
        removed_non_pink.append(fname)
        continue

    cv2.imwrite(os.path.join(CLEANED_DIR, fname), img)
```

```
    kept.append(fname)

print(f"Kept               : {len(kept)} → {kept}")
print(f"Removed (night)    : {len(removed_night)} → {removed_night}")
print(f"Removed (non-pink) : {len(removed_non_pink)} → {removed_non_pink}")
```

```
Kept               : 5 → ['0002.jpg', '0003.jpg', '0008.jpg',
'0010.jpg', '0020.jpg']
Removed (night)    : 2 → ['0001.jpg', '0009.jpg']
Removed (non-pink) : 3 → ['0007.jpg', '0011.jpg', '0030.jpg']
```

Guide Questions — Action Item 2:

1. What mathematical or statistical bases have you considered when developing your function?
- The development of the data cleaning function is based on several mathematical and statistical principles, particularly those related to pixel distribution and classification. For nighttime detection, one key concept used is the mean, specifically the average value of the V (intensities) channel in HSV space, which represents the overall brightness of an image. This aligns with statistical analysis where the mean serves as a measure of central tendency. Thresholding is also applied to this, where predefined cutoff values are used to classify images as either nighttime or daytime based on brightness levels.

On the identification of images with a pink flower, the method uses pixel proportion by computing the ratio of pixels that fall within a specific hue and saturation range, effectively measuring how dominant a color is in the image. This approach reflects probability-based reasoning, where a higher proportion indicates a higher likelihood that the image contains the target feature. HSV color space is particularly suitable for this type of analysis because it separates color information from brightness, making statistical operations on each component more meaningful *(Fiveable, 2026)*.

2. What are the challenges in re-orienting the images in this action item?
- One of the main challenges in re-orienting or processing the images in this action item is the variability in lighting conditions, which significantly affects both brightness and perceived color. The same object may appear different under varying illumination, making it difficult to rely on fixed thresholds. Another challenge is color ambiguity, as pink hues can overlap with red or purple in the HSV spectrum, leading to possible misclassification. Background elements such as leaves, sky, or shadows can also interfere with color detection by introducing additional pixel values that do not belong to the object of interest. Furthermore, differences in image composition, such as occlusion, partial visibility, or varying orientations of the flowers, make it harder to consistently detect the intended features. These issues are compounded by the sensitivity of threshold-based methods, where small changes in parameter values can significantly impact the results. Although HSV improves interpretability, it is still influenced by environmental and imaging conditions.

3. What can you suggest for automating such a task?
- To automate this task more effectively, more advanced and adaptive techniques can be implemented. One approach is to use machine learning models, such as convolutional neural networks, which can learn complex patterns and features from labeled data instead of relying on fixed thresholds. Another improvement is adaptive thresholding, where threshold values are dynamically determined based on the dataset rather than being manually set. Clustering methods such as k-means can also be used to automatically identify dominant colors in images without predefined ranges. Additionally, combining multiple color spaces, such as RGB, HSV, and LAB, can improve robustness by capturing different aspects of color information. Preprocessing techniques like normalization and noise reduction can further enhance consistency before classification. These methods leverage the strengths of HSV, particularly its ability to separate color from brightness,

making automated analysis more reliable and scalable *(Color Spaces in Image Processing, 2023)*.

Action Item 3: Data Enhancement

From the cleaned dataset, develop and apply a function that:
- **Adjusts the exposure** of the images based on the brightest image *(Hint: Gamma Correction)*
- **Adjusts the saturation** of the images to match the saturation of the most saturated image *(Hint: Histogram Equalization + Channel Arithmetic)*

In [ ]:

```
# ==============================
# Inspect Dataset Stats for Manual Selection
# ==============================
# Run this to identify which cleaned image is the brightest and
# which is the most saturated. Then update BRIGHTEST_IMAGE and
# MOST_SATURATED_IMAGE in the config cell (top of notebook) if needed.

cleaned_files = sorted(glob(os.path.join(CLEANED_DIR, '*.jpg')))

print(f"{'Filename':<15} {'Mean Brightness (V)':<25} {'Mean Saturation
(S)'}")
print("-" * 60)

stats = []
for fpath in cleaned_files:
    img   = cv2.imread(fpath)
    hsv   = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    mean_v = float(np.mean(hsv[:, :, 2]))
    mean_s = float(np.mean(hsv[:, :, 1]))
    stats.append((os.path.basename(fpath), mean_v, mean_s))
    print(f"{os.path.basename(fpath):<15} {mean_v:<25.2f} {mean_s:.2f}")

brightest = max(stats, key=lambda x: x[1])
most_sat  = max(stats, key=lambda x: x[2])
print(f"\n→ Brightest image     : {brightest[0]}  (V =
{brightest[1]:.2f})")
print(f"→ Most saturated image: {most_sat[0]}  (S = {most_sat[2]:.2f})")
print("\nUpdate BRIGHTEST_IMAGE / MOST_SATURATED_IMAGE in the config cell
if needed, then re-run.")
```

```
Filename        Mean Brightness (V)       Mean Saturation (S)
------------------------------------------------------------
0002.jpg        153.34                    56.53
0003.jpg        178.13                    138.36
0008.jpg        144.00                    105.90
0010.jpg        135.88                    92.85
0020.jpg        175.76                    61.63

→ Brightest image     : 0003.jpg  (V = 178.13)
→ Most saturated image: 0003.jpg  (S = 138.36)

Update BRIGHTEST_IMAGE / MOST_SATURATED_IMAGE in the config cell if
needed, then re-run.
```

In [ ]:

```
# ==============================
# Gamma Correction (Exposure Adjustment)
# ==============================
# For each image, we compute the gamma value that maps its current mean
brightness
# to the target (brightest image's) mean brightness.

def apply_gamma(img_bgr, gamma):
    """Apply gamma correction to adjust image exposure via a LUT."""
    inv_gamma = 1.0 / gamma
    lut = np.array([((i / 255.0) ** inv_gamma) * 255 for i in range(256)],
dtype=np.uint8)
    return cv2.LUT(img_bgr, lut)

# ==============================
# Saturation Adjustment (Histogram Equalization + Channel Arithmetic)
# ==============================

def adjust_saturation_to_target(img_bgr, target_mean_s):
    """Equalize S channel then scale via channel arithmetic to match target
saturation."""
    hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV).astype(np.float32)
    h, s, v = cv2.split(hsv)

    # Histogram equalization on S channel
    s_eq = cv2.equalizeHist(s.astype(np.uint8)).astype(np.float32)

    # Channel arithmetic: scale equalized S to match target mean
    current_mean = float(np.mean(s_eq))
    if current_mean > 0:
        s_scaled = np.clip(s_eq * (target_mean_s / current_mean), 0, 255)
    else:
        s_scaled = s_eq

    result = cv2.merge([h, s_scaled, v]).astype(np.uint8)
    return cv2.cvtColor(result, cv2.COLOR_HSV2BGR)

# ==============================
# Apply Enhancements to All Cleaned Images
# ==============================
# Outputs are saved to two separate folders so each step can be compared:
#   enhanced_dataset/exposure/   ← gamma correction only
#   enhanced_dataset/saturation/ ← gamma + saturation adjustment (final)

ref_bright = cv2.imread(os.path.join(CLEANED_DIR, BRIGHTEST_IMAGE))
ref_sat    = cv2.imread(os.path.join(CLEANED_DIR, MOST_SATURATED_IMAGE))

if ref_bright is None or ref_sat is None:
    print("ERROR: Could not load representative image(s). Check filenames
in the config cell.")
else:
    target_v = float(np.mean(cv2.cvtColor(ref_bright, cv2.COLOR_BGR2HSV)[:,
:, 2]))
```

```
    target_s = float(np.mean(cv2.cvtColor(ref_sat,    cv2.COLOR_BGR2HSV)[:,
:, 1]))
    print(f"Target brightness (V) from '{BRIGHTEST_IMAGE}'     :
{target_v:.2f}")
    print(f"Target saturation (S) from '{MOST_SATURATED_IMAGE}':
{target_s:.2f}\n")

    cleaned_files = sorted(glob(os.path.join(CLEANED_DIR, '*.jpg')))
    for fpath in cleaned_files:
        img   = cv2.imread(fpath)
        fname = os.path.basename(fpath)

        cur_v = float(np.mean(cv2.cvtColor(img, cv2.COLOR_BGR2HSV)[:, :,
2]))
        eps   = 1e-7
        if cur_v > 0 and target_v > 0:
            gamma = np.log(cur_v / 255.0 + eps) / np.log(target_v / 255.0 +
eps)
            gamma = float(np.clip(gamma, 0.1, 5.0))
        else:
            gamma = 1.0

        img_exposed = apply_gamma(img, gamma)
        cv2.imwrite(os.path.join(ENHANCED_EXP_DIR, fname), img_exposed)

        img_enhanced = adjust_saturation_to_target(img_exposed, target_s)
        cv2.imwrite(os.path.join(ENHANCED_SAT_DIR, fname), img_enhanced)

        print(f"  {fname}: gamma={gamma:.3f}")

    print(f"\nStep 1 (exposure only) → '{ENHANCED_EXP_DIR}'")
    print(f"Step 2 (+ saturation)  → '{ENHANCED_SAT_DIR}'")
```

```
Target brightness (V) from '0020.jpg'     : 175.76
Target saturation (S) from '0003.jpg': 138.36

  0002.jpg: gamma=1.367
  0003.jpg: gamma=0.964
  0008.jpg: gamma=1.536
  0010.jpg: gamma=1.691
  0020.jpg: gamma=1.000

Step 1 (exposure only) → 'A3_enhanced/exposure'
Step 2 (+ saturation)  → 'A3_enhanced/saturation'
```

Action Item 4: Modified Data Augmentation

Reusing the geometric augmentation functions from MP1, with the addition of:
- **Saturation Increase** — multiply the S channel by `1.5×`
- **Saturation Decrease** — multiply the S channel by `0.5×`

All augmentations are applied to the **enhanced dataset**.

In [131]:

```
# ==============================
# Geometric Augmentation Functions (Reused from MP1)
# ==============================

def apply_black_patch(image, patch_size=(20, 20)):
    """Randomly place a black square patch anywhere on the image."""
    h, w = image.shape[:2]
    x = random.randint(0, max(0, w - patch_size[0]))
    y = random.randint(0, max(0, h - patch_size[1]))
    patched = image.copy()
    patched[y:y+patch_size[1], x:x+patch_size[0]] = 0
    return patched

def shift_image(image, tx, ty):
    """Translate image by (tx, ty) pixels using an affine warp."""
    h, w = image.shape[:2]
    M = np.float32([[1, 0, tx], [0, 1, ty]])
    return cv2.warpAffine(image, M, (w, h))

def rotate_image(image, angle):
    """Rotate image by the given angle (degrees) around its center."""
    h, w = image.shape[:2]
    M = cv2.getRotationMatrix2D((w // 2, h // 2), angle, 1.0)
    return cv2.warpAffine(image, M, (w, h))

def flip_image(image, mode):
    """Flip image: 'horizontal' (left-right) or 'vertical' (top-bottom)."""
    return cv2.flip(image, 1 if mode == 'horizontal' else 0)

# ==============================
# New: Saturation Augmentation
# ==============================

def adjust_saturation(image, scale):
    """
    Scale the S channel by the given factor.
    scale > 1 → increase saturation (more vivid colors)
    scale < 1 → decrease saturation (muted/desaturated colors)
    """
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV).astype(np.float32)
    h, s, v = cv2.split(hsv)
    s = np.clip(s * scale, 0, 255)
    result = cv2.merge([h, s, v]).astype(np.uint8)
    return cv2.cvtColor(result, cv2.COLOR_HSV2BGR)

# ==============================
# Apply All Augmentations to Enhanced Dataset (saturation/ folder = final
output)
# ==============================
enhanced_files = sorted(glob(os.path.join(ENHANCED_SAT_DIR, '*.jpg')))
enhanced_files2 = sorted(glob(os.path.join(ENHANCED_EXP_DIR, '*.jpg')))

if not enhanced_files:
```

```
    print("No enhanced images found. Run Action Item 3 first.")
else:
    print(f"Processing {len(enhanced_files + enhanced_files2)} images...")

    # For Enhanced Saturation Dataset
    for fpath in enhanced_files:
        img      = cv2.imread(fpath)
        raw_name = os.path.splitext(os.path.basename(fpath))[0]

        # MP1 geometric augmentations
        cv2.imwrite(os.path.join(BASE_AUG_DIR, 'patch',
f'patch_{raw_name}.jpg'), apply_black_patch(img, P_SIZE))
        cv2.imwrite(os.path.join(BASE_AUG_DIR, 'shift',
f'shift_{raw_name}.jpg'), shift_image(img, S_TX, S_TY))
        cv2.imwrite(os.path.join(BASE_AUG_DIR, 'rotate',
f'rot_{raw_name}.jpg'),   rotate_image(img, ROT_A))
        cv2.imwrite(os.path.join(BASE_AUG_DIR, 'flip',
f'flip_{raw_name}.jpg'),  flip_image(img, FLIP_A))

        # New: saturation augmentations
        cv2.imwrite(os.path.join(BASE_AUG_DIR, 'sat_increase',
f'satup_{raw_name}.jpg'),   adjust_saturation(img, SAT_INCREASE))
        cv2.imwrite(os.path.join(BASE_AUG_DIR, 'sat_decrease',
f'satdown_{raw_name}.jpg'), adjust_saturation(img, SAT_DECREASE))

    # For Enhanced Exposure Dataset
    for fpath in enhanced_files2:
        img      = cv2.imread(fpath)
        raw_name = os.path.splitext(os.path.basename(fpath))[0]

        # MP1 geometric augmentations
        cv2.imwrite(os.path.join(BASE_AUG_DIR2, 'patch',
f'patch_{raw_name}.jpg'), apply_black_patch(img, P_SIZE))
        cv2.imwrite(os.path.join(BASE_AUG_DIR2, 'shift',
f'shift_{raw_name}.jpg'), shift_image(img, S_TX, S_TY))
        cv2.imwrite(os.path.join(BASE_AUG_DIR2, 'rotate',
f'rot_{raw_name}.jpg'),   rotate_image(img, ROT_A))
        cv2.imwrite(os.path.join(BASE_AUG_DIR2, 'flip',
f'flip_{raw_name}.jpg'),  flip_image(img, FLIP_A))

        # New: saturation augmentations
        cv2.imwrite(os.path.join(BASE_AUG_DIR2, 'sat_increase',
f'satup_{raw_name}.jpg'),   adjust_saturation(img, SAT_INCREASE))
        cv2.imwrite(os.path.join(BASE_AUG_DIR2, 'sat_decrease',
f'satdown_{raw_name}.jpg'), adjust_saturation(img, SAT_DECREASE))

    print(f"Done. Augmented images saved to '{BASE_AUG_DIR}/' and
'{BASE_AUG_DIR2}/'")
    print(f"  Subfolders: patch | shift | rotate | flip | sat_increase |
sat_decrease")
```

```
Processing 10 images...
Done. Augmented images saved to 'A4_augmented/exposure/' and
```

```
'A4_augmented/saturation/'
  Subfolders: patch | shift | rotate | flip | sat_increase |
sat_decrease
```

References

Color spaces in image processing. (2023, July 12). Scaler Topics. https://www.scaler.com/topics/color-spaces-in-image-processing/

Fiveable. (2026, March 4). 2.1 Histogram manipulation – Computer Vision and Image Processing. https://fiveable.me/computer-vision-and-image-processing/unit-2/histogram-manipulation/study-guide/hkUGJHwLZZyfHqu8

Difference between rgb, cmyk, hsv, and yiq color models. (2021, November 16). GeeksforGeeks. https://www.geeksforgeeks.org/computer-graphics/difference-between-rgb-cmyk-hsv-and-yiq-color-models/

Hsv color model in computer graphics. (2022, August 20). GeeksforGeeks. https://www.geeksforgeeks.org/computer-graphics/hsv-color-model-in-computer-graphics/

HSV (Hue, Saturation, and Value). (n.d.). Retrieved April 9, 2026, from https://yzqhl.intantles.my.id/help/?32161731835980&extra_param_1=d7bs2ad3kl6c73fipa50

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.